# Pretraining Lecture #1

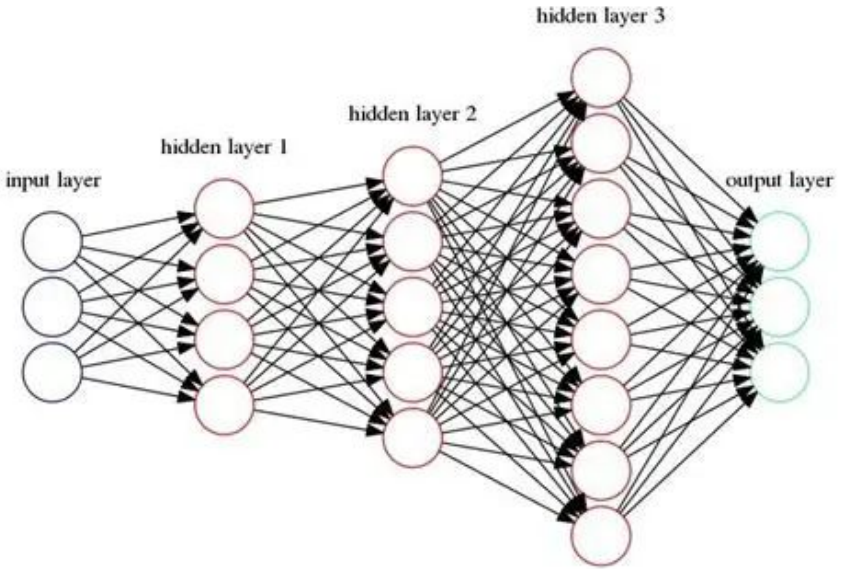
## 1/27/2026

Archit Kumar, Divya Pothavajhyula, Maaz Hussain, Rohan Tolani

# Zero Bubble (Almost) Pipeline Parallelism

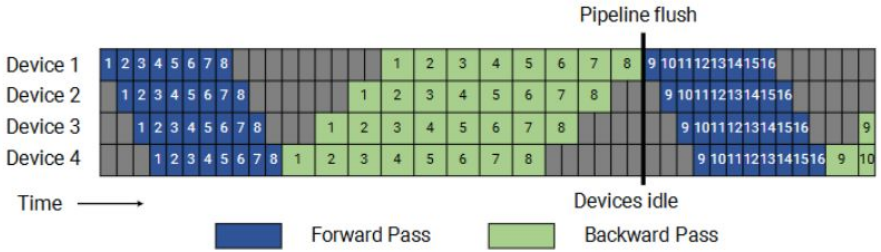
# Background: Naive Pipeline Parallelism

- Assigns contiguous layers of the model to different GPUs
- Example: GPU #1 gets Input → Hidden Layer 1, GPU #2 gets Hidden Layer 1 → Hidden Layer 2, and so on
- Issue: processing one batch at a time = most GPUs are idle
- Solution: Split a batch into microbatches, stream them through the pipeline



# Background: Microbatches

- Split a batch into **m** microbatches
- Leads to dependencies:
  - Forward on GPU #3 can't start until GPU #2 finishes forward
  - Backward on GPU #3 can't start until GPU #4 finishes backward
- Pipeline Bubble: time when a GPU is idle due to dependencies
- Utilization: 1 - (p - 1)/m
  - p = number of pipeline stages
- More microbatches means fewer bubbles, but more activation memory



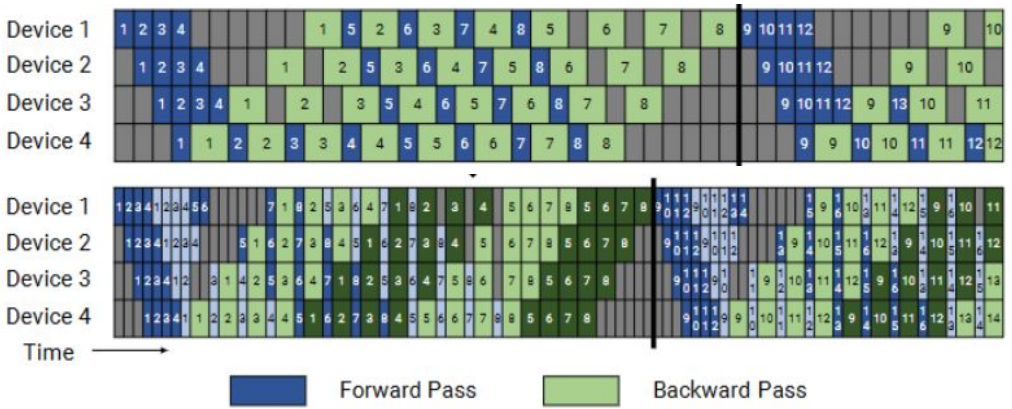

GPipe Schedule: Runs all forwards, then all backwards. Large bubbles, high memory usage

# Background: One-Forward-One-Backward (1F1B)

- Idea: As soon as a microbatch finishes forward, start its backward ASAP
- Reduces activation memory
- Issue: Still has bubbles

# Background: Interleaved 1F1B

- Instead of each GPU owning one contiguous chunk of layers, have them own multiple smaller chunks, interleaved across pipeline
  - E.g. instead of GPU #1 having layers 1-4, GPU #1 has layers 1-2 and 9-10
  - If GPU #1 is waiting on chunk 1, it can now switch to chunk 2 and keep computing
- Pipeline flush happens sooner, bubble size reduced by a factor of v(number of interleaved chunks per GPU)
- Communication increases by a factor of v
- Higher peak memory




1F1B (top) vs Interleaved 1F1B(bottom). Bubble size reduced by a factor of v(2) since splitting each stage into multiple interleaved chunks shortens and overlaps pipeline flush periods.

# Background: Synchronous vs. Asynchronous Training

Synchronous:

1. Every GPU does F + B
2. Gradients are synchronized
3. Synchronization barrier (i.e. waiting)
4. Optimizer step
5. Next iteration

All GPUs always on same iteration, so deterministic and stable convergence

Asynchronous:

1. GPUs compute gradients on their own data (independently)
2. Gradients applied whenever they are ready
3. Optimizer step (using potentially stale or partial gradients)

GPUs not always on same iteration, so theoretically zero bubble, but at the cost of convergence guarantees / stale gradients

# Background: Synchronous vs. Asynchronous Training

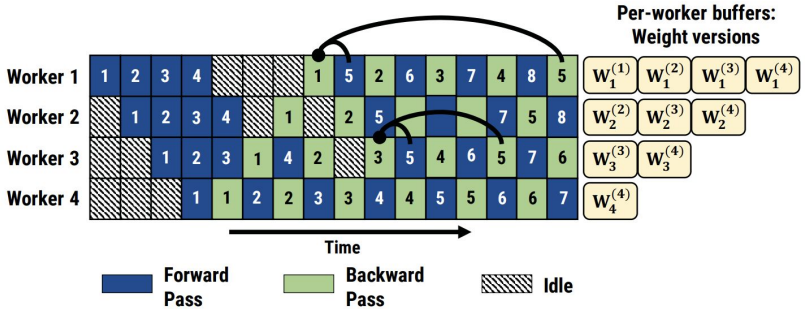



**Figure 9: Weight stashing as minibatch 5 flows across stages. Arrows point to weight versions used for forward and backward passes for minibatch 5 at the first and third stages.**

Asynchronous:

1. GPUs compute gradients on their own data (independently)
2. Gradients applied whenever they are ready
3. Optimizer step (using potentially stale or partial gradients)

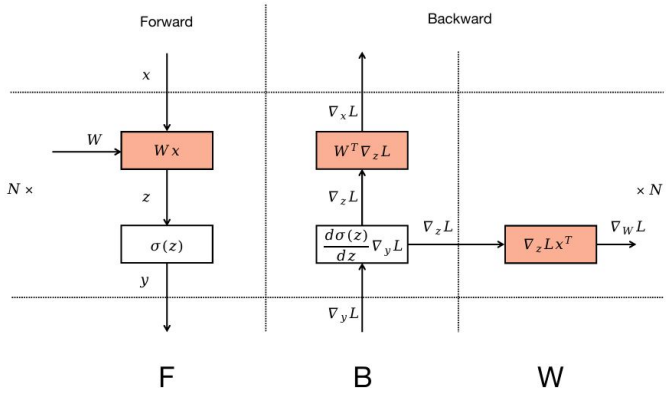
GPUs not always on same iteration, so theoretically zero bubble, but at the cost of convergence guarantees / stale gradients

# Key Takeaways from Megatron Paper

- Pipeline parallelism is essential at scale
- 1F1B is the right baseline
- Interleaving helps but:
  - Costs Memory
  - Costs communication
  - Reduces bubbles, but does not eliminate them
- Bubbles were widely treated as inevitable under synchronous training
- Megatron's key assumption: **The backward pass is 1 indivisible operation**

# Zero Bubble's Trick

- The backward pass is composed of two computations:
  - **B:** the gradient with respect to the input **x**
  - **W:** the gradient with respect to the layer's weights
  - **F:** Forward pass
- Issue before: Stage 1's **B** would be blocked waiting for Stage 2's **W** to finish, even though Stage 1 only needs the activation gradient signal (**B**)
- **F** and **B** still have strict dependencies
- **W** can be delayed/moved around, as long as it happens after **B**




**Forward and backward pass. The weight matrix *W* is used in each step: in F to compute activations, in B to compute activation gradients, and in W to compute the weight gradient.**

# Handcrafted Schedules: ZB-H1 and ZB-H2

ZB-H1 (Not exceeding peak memory of 1F1B):

- Mimics 1F1B
- **B** starts earlier across workers
- Late **W** computations fill the tail bubbles
- Memory stays similar because **W** needs less memory than **B**, and the max memory still occurs towards the beginning
- ⅓ pipeline bubble size compared to 1F1B

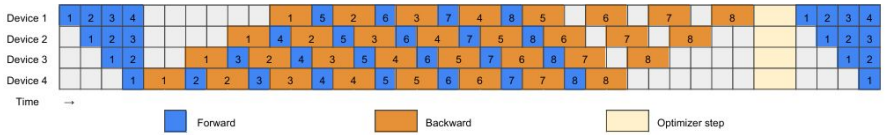



Figure 2: 1F1B pipeline schedule.


**ZB-H1 schedule**

# Handcrafted Schedules: ZB-H1 and ZB-H2

ZB-H2 (Not limited to peak memory of 1F1B):

- Adds more forward passes during warmup to eliminate early bubbles before the first backward
- Reorders **W** at the tail so the shape becomes a parallelogram
- Caveat: removes optimizer-step synchronization

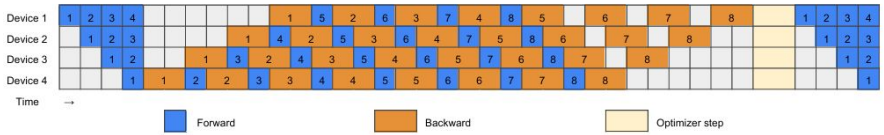



Figure 2: 1F1B pipeline schedule.

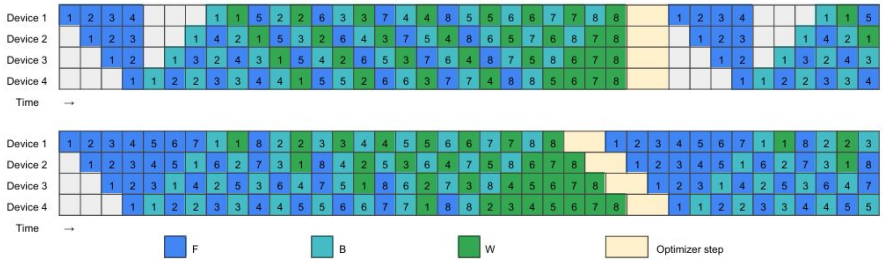



Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

**Note: ZB-H2 has a much higher peak memory compared to 1F1B and ZB-H1, due to its elimination of early bubbles from running more forward passes during warmup.**

# Handcrafted Schedules - Quantitative Analysis

ZB-H1 and ZB-H2 rely on standard properties of transformer training:

- Runtime of **W** < Runtime of **F** < Runtime of **B**
- **B** needs lot of activation storage, **W** needs less
- Bubble size of ZB-H2 < Bubble size of ZB-H1 < Bubble size of 1F1B
- Peak activation memory is highest for ZB-H2
- We can buy bubble size reduction with memory

# Automatic Pipeline Scheduling: Motivation

- TF, TB, TW have different runtimes
- TComm (inter-stage communication time) is non-negligible
- Memory limit may prevent enough microbatches for a bubble-free schedule
- Handcrafted schedules break under realistic conditions

Goal: Automatically find an optimal (or near-optimal) schedule under these constraints

- Heuristic algorithm for scalability provides an optimal/near optimal schedule
- Integer Linear Programming for small-scale refinement

# Automatic Scheduling: High Level

- Aggressively reduce bubbles during warm up
- Maintain steady state pipeline flow
- Insert W updates to:
  - Fill bubbles
  - Recycle memory
- Constraints: stage dependencies & memory limits

# Automatic Pipeline Scheduling Algorithm

1. Warm up: schedule as many **F** as possible
    - Goal: shrink bubble before first **B**
    - Use hyperparameter to decide whether inserting extra **F** is worth delaying **B**
2. Steady state:
    - Follow **1F1B** pattern
    - Insert **W** to fill gaps
        - Gap >= TW
        - Gap < TW but reduces worst-stage bubble
        - Memory limit is hit (**W** will free activations)
    - Stage i must always be >=1 **F** stage ahead of stage i + 1
        - If difference > 1, use hyperparameter to decide whether to skip **F**
3. Drain phase: when **F** and **B** are done, schedule remaining **W**

Use Grid Search to find best hyperparameters

# Bypassing Optimizer Synchronizations

- Previous implementations do global synchronization at optimizer step in order to clip gradients and check for Nan/Inf (from mixed precision)
- All-reduce to collect the global states, followed by the optimizer steps
- Optimizer-step sync says to wait for the slowest GPU, reintroducing pipeline bubbles
- Idea: post-update validation instead of pre-update synchronization
    - NaN/Inf and gradient clip rates are rare, so sync wastes time

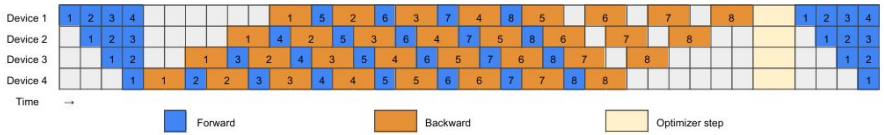



Figure 2: 1F1B pipeline schedule.

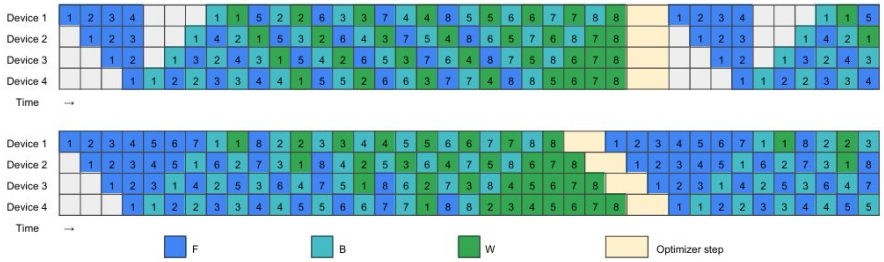
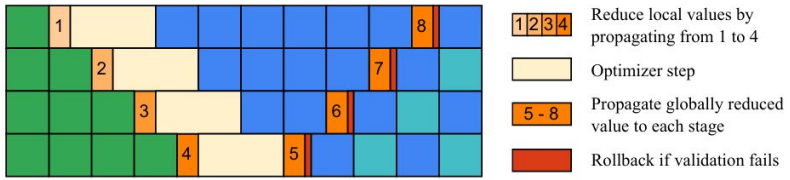



Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

# Bypassing Optimizer Synchronizations

- Stage 1 computes local state
- Stage 1 sends to Stage 2, which combines with its local state
- Stage 2 sends to Stage 3…
- Each stage performs optimizer update based on the partial reduced state it has
- During warmup of next iteration, the fully reduced global state is sent back from last stage to first
- Each stage then validates to determine if its previous optimizer step was legitimate

Issue: What if the optimizer step was illegitimate?



# Bypassing Optimizer Synchronizations - Rollback

- Rollback is expensive (storage of old parameters)
- In-place rollback
  - Some optimizer steps are arithmetically reversible
- Avoids unneeded copies, and only pays extra compute when necessary

**Algorithm 1** In-place rollback for AdamW

**Optimizer States:**
  $\gamma$(lr), $\beta_1, \beta_2$(betas), $\epsilon$ (epsilon), $\lambda$(weight decay),
  $m$ (first moment), $v$ ( second moment), $\theta$ (parameters),
  $t$(time stamp).
**function** STEP($g$) ▷ In-place step
  $t = t + 1$
  $m = \beta_1 m + (1 - \beta_1) g$
  $v = \beta_2 v + (1 - \beta_2) g^2$
  $m' = m / (1 - \beta_1^t)$
  $v' = v / (1 - \beta_2^t)$
  $\theta = \theta - \gamma\lambda\theta - \gamma m' / (\sqrt{v'} + \epsilon)$
**end function**
**function** ROLLBACK($g$) ▷ In-place rollback
  $m' = m / (1 - \beta_1^t)$
  $v' = v / (1 - \beta_2^t)$
  $\theta = (\theta + \gamma m' / (\sqrt{v'} + \epsilon)) / (1 - \gamma\lambda)$
  $m = (m - (1 - \beta_1) g) / \beta_1$
  $v = (v - (1 - \beta_2) g^2) / \beta_2$
  $t = t - 1$
**end function**

# Experiments

- Hardware: 32 NVIDIA A100 GPUs across 4 nodes
- Models: GPT-3 like architectures (1.5B - 28.3B params)
- 1F1B, ZB-1p, ZB-2p all bit-to-bit identical
- ZB-1p outperforms 1F1B in multi-node setups (TComm non-negligible)
- ZB-2p consistently has highest throughput (up to 30% higher than 1F1B) even with fewer microbatches

Table 3: Models and fixed settings used in experiments

| Model | Layers | Attention Heads | Hidden Size | Sequence Length | Pipelines (GPUs) | Microbatch Size | Number of Microbatches |
|---|---|---|---|---|---|---|---|
| 1.5B | 22 | 24 | 2304 | 1024 | 8 | 6 | 24 / 32 / 64 |
| 6.2B | 30 | 32 | 4096 | 1024 | 8 | 3 | 24 / 32 / 64 |
| 14.6B | 46 | 40 | 5120 | 1024 | 16 | 1 | 48 / 64 / 128 |
| 28.3B | 62 | 48 | 6144 | 1024 | 32 | 1 | 96 / 128 / 256 |

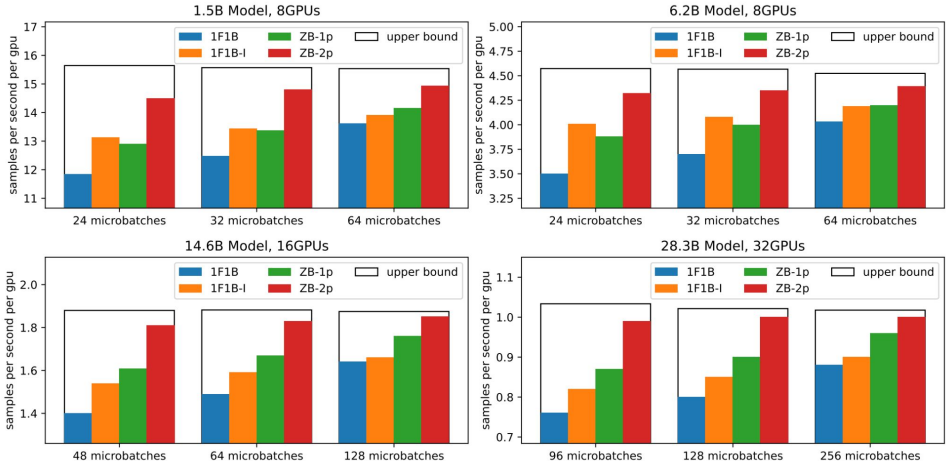



Figure 5: Comparison of throughput across different pipeline schedules.

# Experiments

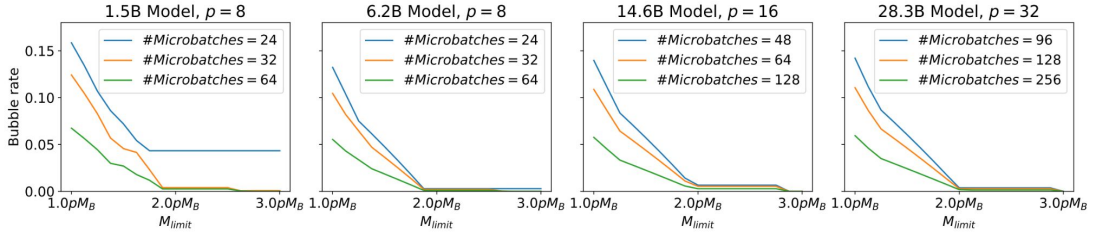



Figure 7: The relation between memory limit and bubble rate using our heuristic algorithm.

- Bubble rate: (largest execution time - optimal execution time) / largest execution time
- Most cases: ZB-p2 has bubble rate <1%
- ZB-p2 consistently outperforms ZB-H2 → shows automatic pipeline adapts better to realistic scenarios
- Linear relationship between MLimit and bubble rate; after inflection point, diminishing returns
- 2pMB 'best' threshold to achieve zero bubble rate when TF ~ TB, TComm relatively small

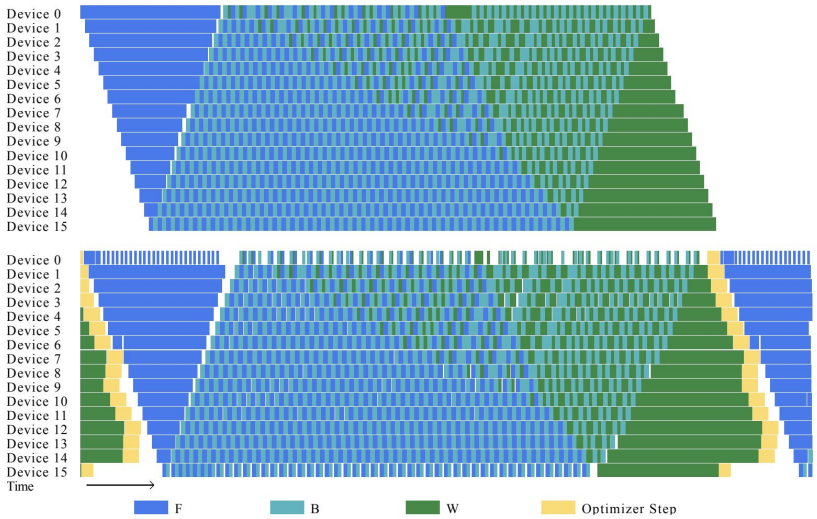



Figure 6: A schedule produced by ZB-2p (top) and its profiled execution process (bottom).

# Weaknesses & Future Directions

| Weaknesses: | | Future Direction: |
| --- | --- | --- |
| Requires about double the activation memory of 1F1B to truly reach zero bubble | → | Utilize ZeRO, tensor parallelism, or memory-adaptive scheduling |
| Relies on accurate profiling (i.e. not robust against T values changing over time) | → | Implementing periodic re-profiling during training to ensure accurate T values |
| Potential rollback during optimizer step is expensive | → | Perform partial rollback to only the affected stages |
| Unstable models with high LR warm-up phase will trigger many rollbacks | → | Disable optimizer sync bypass during unstable phases |

# Related Work & Strengths of Zero Bubble

Baseline (1F1B) from Megatron-LM paper:

- Reduces bubbles via model structure
- Assumes uniform compute per layer
- Requires many microbatches
- Limited flexibility (fixed structure)
- Synchronous training

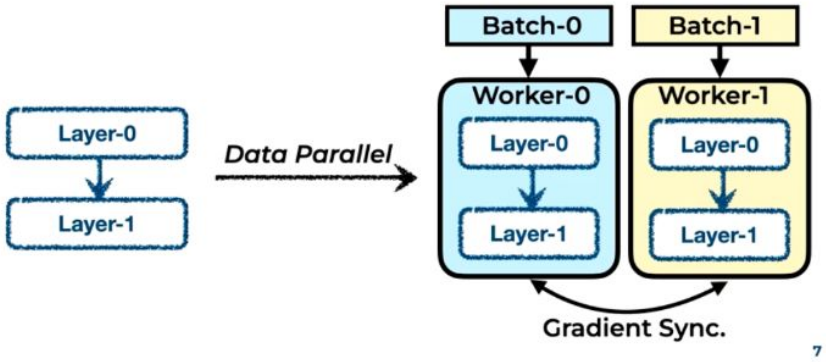
Strengths of ZB:

- Reduces bubbles via automatic scheduling
- Schedules using profiled T values
- Works even with fewer microbatches
- Reorders F, B, W dynamically
- Synchronous training
- Bit-to-bit identical results to 1F1B

# WLB-LLM: Workload-Balanced 4D Parallelism for LLM Training

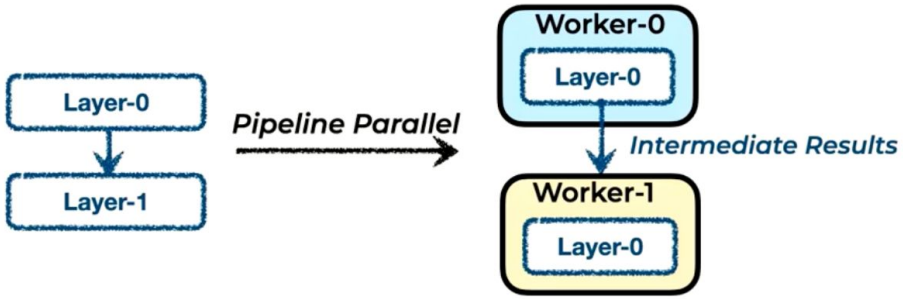
# Background: 4D Parallelism - Data Parallelism

- Make multiple replicas of the model, (Worker-0 and Worker-1 hold the same model)
- Each DP replica processes different training examples and replicas synchronize gradients so everyone applies the same update.
- Scales training throughput by increasing effective batch size



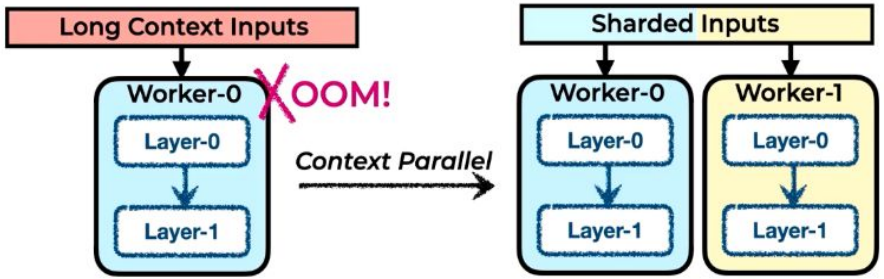
# Background: 4D Parallelism - Pipeline Parallelism

- Split the model's layers across different GPUs and split the batch into smaller pieces so different GPUs can work on different micro-batches simultaneously.
- Worker 0 holds earlier layers and sends intermediate activations to Worker 1 during the forward pass and gradients flow back the other way in the backwards pass.
- This allows you to train models that don't fit on one GPU by reducing per-GPU memory



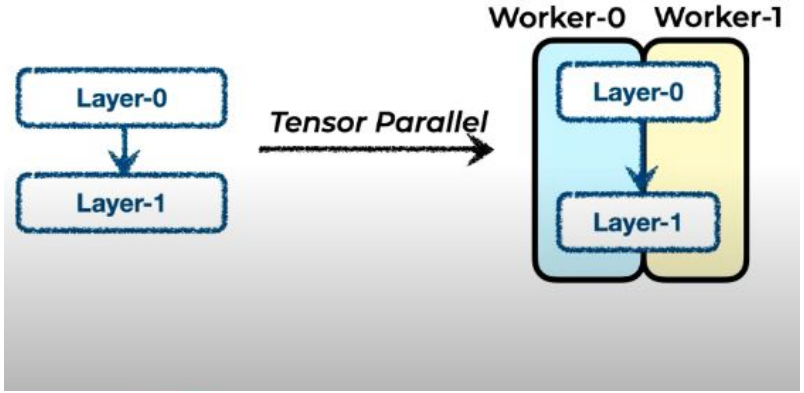
# Background: 4D Parallelism - Context Parallelism

- Splits a long input sequence across multiple GPUs so each GPU handles only a chunk of the tokens
- Cuts activation memory because each GPU stores only its token-chunk's activation - this prevents OOM issues with long contexts
- During attention, the GPUs share the needed key/value information
- Distinct from Sequence Parallelism which only shards the sequence for a few layers usually as an efficiency tactic for TP

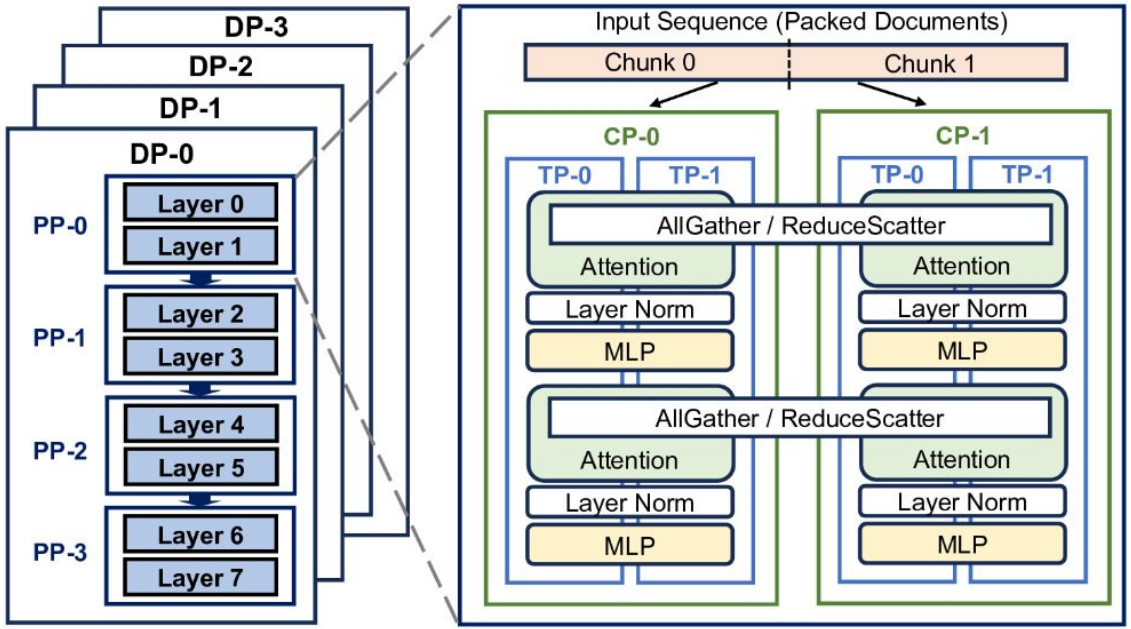


# Background: 4D Parallelism - Tensor Parallelism

- Split the model parameters within a single layer across multiple GPUs
- Big matrix multiplications are thus split across GPUs which makes individual layers feasible and faster by distributing huge matrix multiplications.
- After partial results, TP GPUs use collectives to assemble the correct output.

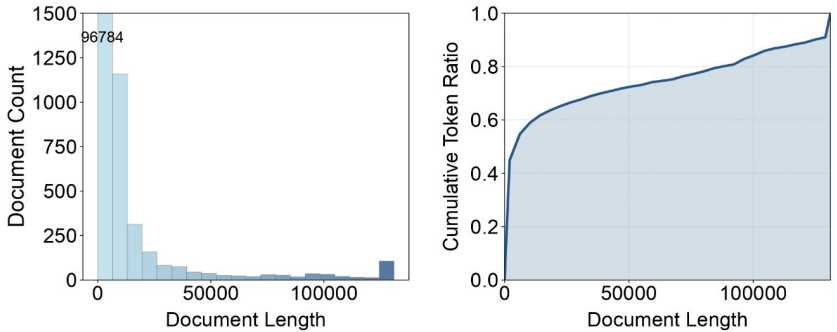


# Background: 4D Parallelism Diagram



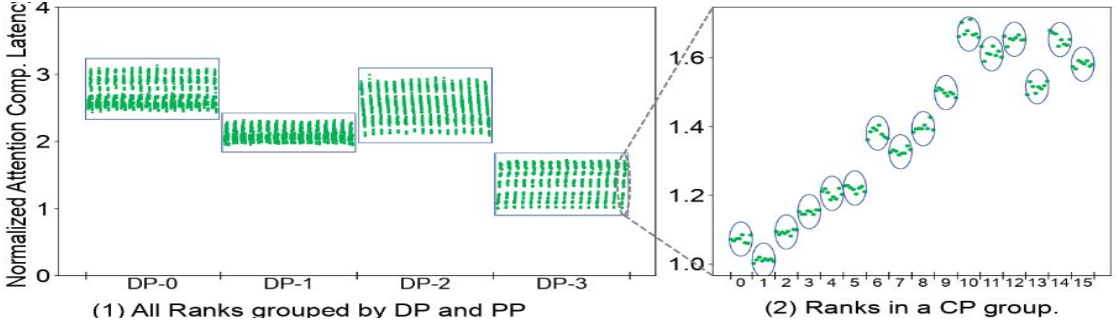
# Motivation: Workload Imbalance in LLM Training

- As LLMs are increasingly used applications, computing needs are growing
- During training GPUs often spend a ton of time sitting idle (slowest GPU latency 1.44x longer)
- Root cause: tokens require very different amounts of attention
    - Even if tokens are divided evenly, tokens in long documents are more expensive
- Today's distributed LLM training stacks balance token counts but not token compute.



# Imbalance Analysis across pipeline

- Measured attention computation latency per GPU on a 8K-GPU, 128K window for 405B model
    1. Attention latency varies greatly across DP workers not PP workers inside DP workers — indicates the problem is with micro-batches having varying compute
    2. Latency varies greatly across CP workers, but TP workers inside each CP worker look similar — indicates the problem is with how the sequence is split across CP workers
- Thus, WLB-LLM fixes imbalance at two levels: the CP-level and the DP-level. Training is synchronized with collectives so the latency is determined by the slowest worker

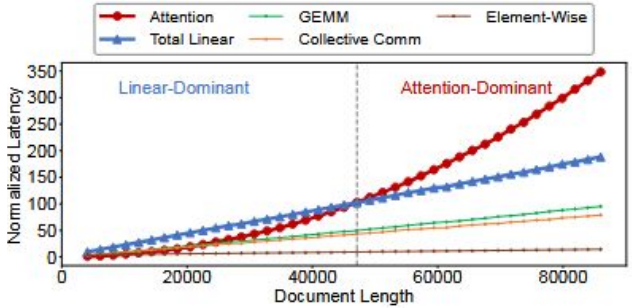


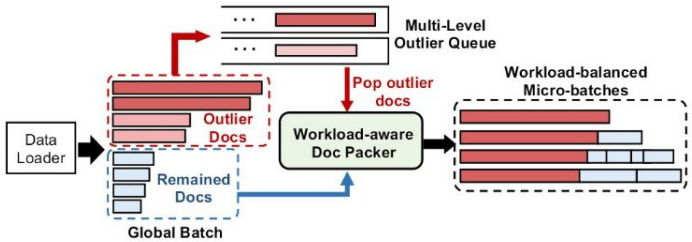
# Baseline Solution DP-level: Fixed-length packing

- Goal: Optimize how docs are packed into micro-batches while keeping a fixed sequence length
- Attention cost for a micro-batch is proportional to the sum of squared doc lengths inside them
- Finding the optimal packing under these constraints is an NP-hard problem.
- Formulate it as an ILP problem where each doc is assigned to one micro-batch with a set token length amount seeking to minimize micro-batch workload
- BUT fixed-length packing can’t get good workload balance without compromising model quality

# Design: Workload-Balanced Pipeline Parallelism

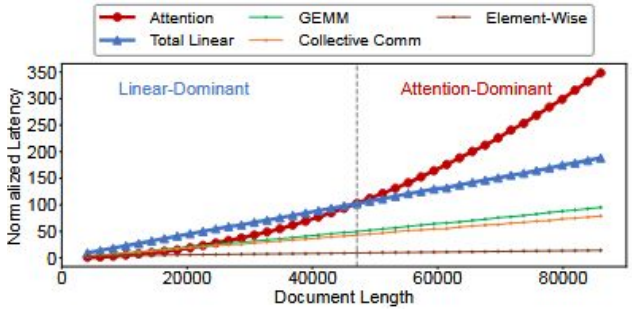
- Traditional Pipeline Parallelism suffer from idle time, caused when micro-batches have different sequence lengths
- WLB-LLM proposes a **balanced micro-batch partitioning strategy**
- Analyze the computation cost of sequences and organizes them so each pipeline stage has the same workload
- Traditional PP just looks at amount of tokens, WLB-LLM takes into account the lengths of the documents, and how **tokens at the end of long documents have much greater latency** because of additional attention
- This significantly decreases the PP idle time

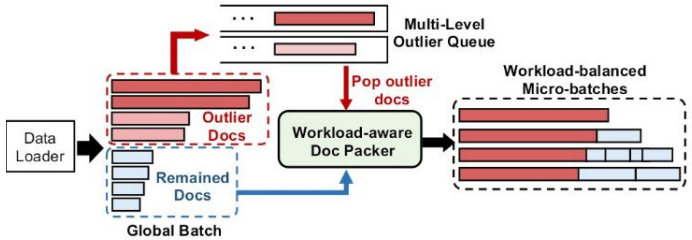


# Design: Outlier Document Delay

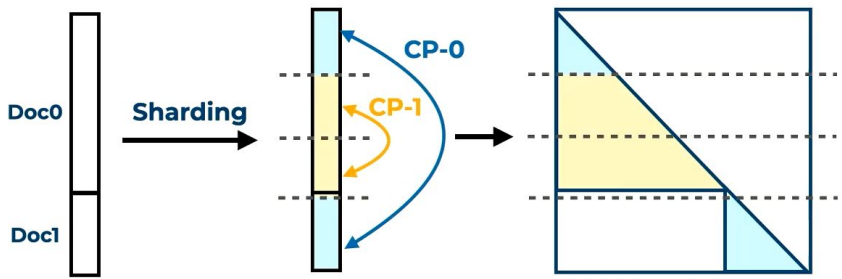
- Extremely long, outlier documents can increase the time for a micro-batch so much, the entire cluster idles
- Long documents cause the most imbalance **but they represent only a small proportion of total tokens**
- By delaying outliers WLB-LLM better balances with negligible impact on the data loading randomness and model convergence
- When a new batch arrives extremely long documents are flagged as outliers
- Outliers are placed into multi-level waiting queue, and once there are enough outliers to divide among the micro-batches WLB-LLM distributes them

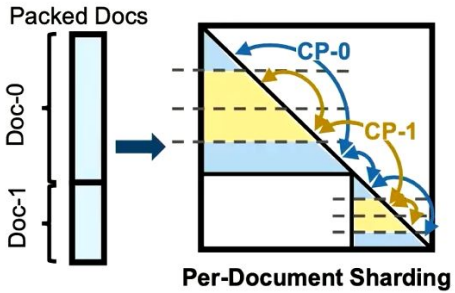


# Design: Fine-grained Context Parallelism Balancing

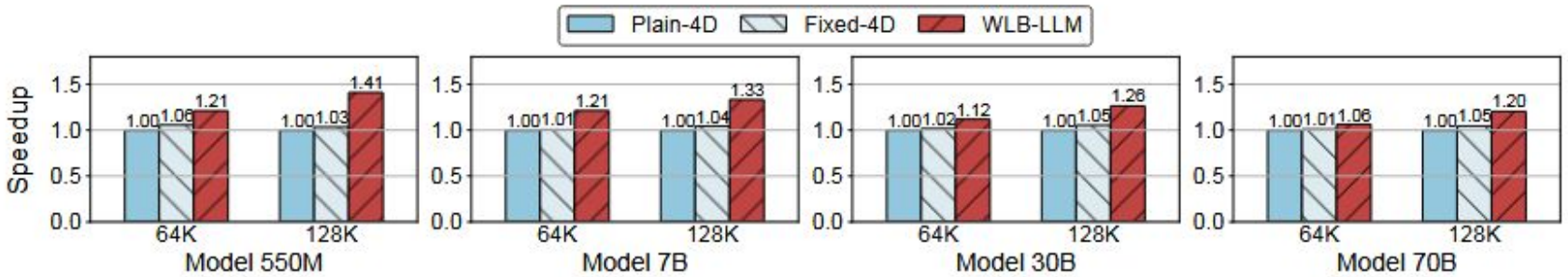
- Existing context parallelism uses sharding to divided documents to each compute unit when inputting
- But because of aforementioned attention complexity, straightforward sharding can lead to an imbalance
- WLB-LLM implements a fine-grained approach, where we do document-level partition
- This optimizes the amount of KV computations across workers



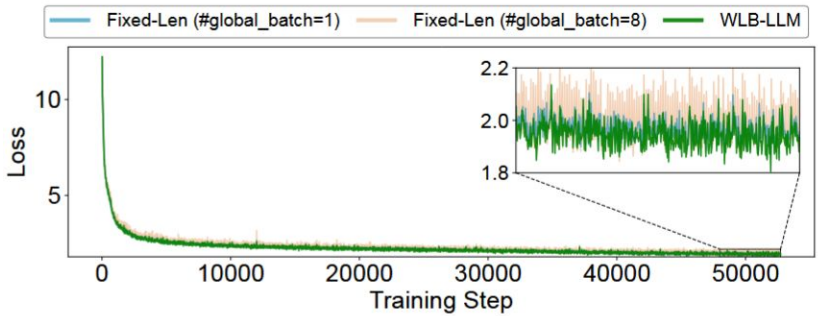
# Results

- Tested on large-scale clusters (256 total NVIDIA H100s) training large models ranging from 550M to 70B parameters
- Compared against Fixed-4D and Plain-4D, internal Llama 3 training frameworks (where they implement baseline approaches to parallelism such as fixed-length packing)



# Conclusion

- 1.23x average speedup across the SOTA used in Llama 3, **without compromising the quality of the resulting models**
- **Addresses the major limitations of 4D parallelism**
- Fine grained sharding can sometimes lead to smaller chunks that reduce the hardware utilization (kernel efficiency) of the GPU
- Future work could extend these balancing technologies to other types of model training, such as addressing routing imbalances in MoE (Mixture of Experts)



# Thank You

# ZB Questions

- Necessary to have zero bubbles?
  - Not strictly, but at a large scale, it's impact really shows (at large scales small bubbles are amplified + TComm grows)
- Dealing with unstable models causing many rollback operations
  - Not bypass optimizer sync stage during this phase of training
- Will ZB still work for modern architectures (FlashAttention, MoE, larger models)?
  - Mostly yes, but MoE likely needs dynamic or online scheduling
- Could errors accumulate when rollbacks occur due to floating-point calculations having precision limits?
  - Rollbacks are rare and ZB-2p is bit-to-bit identical to 1F1B, so practically no; however, in theory, yes

# ZB Questions

- Why is idling optimal when gap < TW and not the worst-stage bubble?
  - Technically, it's not globally optimal, rather is locally under the objective of minimizing runtime of slowest stage
- When is double MLimit of using ZB-2p justified?
  - When you are compute or communication bound, doing multi-node training, have large batches / long runs
- Does heuristic scheduling add complexity that could be a downside?
  - Technically, in terms of code complexity or debugging, but overall, it results in better resource utilization
- Does ZB only work with 2 chunks? Can it be expanded?
  - Yes, but could lead to much more complexity and memory usage

# ZB Questions

- How does splitting B and W affect the critical path?
  - Reduces dependencies that block each other and allows W to run at ‘convenient’ times
- How realistic is the “relaxed memory constraint”?
  - Somewhat, but can combat with ZeRO, TP, or memory-adaptive scheduling
- How much do ZeRO / tensor parallelism help with increased memory cost?
  - Significantly: ZeRO reduces gradients and TP reduces activations per GPU
- Are DualPipe and ZB-PP orthogonal?
  - Yes, DualPipe uses bidirectional chunks and ZB is splitting B + scheduling
- Does ZB substantially increase risk of OOM errors?
  - Potentially, since ZB increases peak memory usage, so things like adaptive scheduling should be employed

# ZB Questions

- Can noise in GPU timing lead to resurgence of bubbles?
  - Yes, but implementing feedback-based scheduling can combat this
- If rollbacks happen often, does ZB lose its benefit?
  - Disabling rollbacks during unstable training phases would help; in general though, rollbacks are rare
- What other framework design decisions are limiting training efficiency?
  - Strict ‘waiting’ barriers / frequent synchronization
- How accurate are profiled times vs real training?
  - Usually accurate over ‘shorter’ lengths, but can combat with re-profiling during training